Next, we will use MLflow to log and serve the model via Databricks Model Serving.

```
import mlflow
# Set the registry URI to "databricks-uc" to configure
# the MLflow client to access models in UC
mlflow.set_registry_uri("databricks-uc")
model_name = f"{catalog}.{database_name}.cvops_model_mlaction"

from mlflow.models.signature import infer_signature,set_signature
img = df_input['data']
predict_sample = df[['label', 'labelName']]
# To register models under UC you require to log signature for both
# input and output
signature = infer_signature(img, predict_sample)

print(f"Your signature is: \n {signature}")

with mlflow.start_run(run_name=model_name) as run:
    mlflowModel = mlflow.pyfunc.log_model(
        artifact_path="model",
        python_model=wrapper,
        input_example=df_input,
        signature=signature,
        registered_model_name=model_name,
    )
##Alternatively log your model and register later
# mlflow.register_model(model_uri, "ap.cv_ops.cvops_model")
```

Figure 7.34 – Logging and running models using MLflow

During the production phase, you will usually operate on the Aliases and the latest model version, so here we simulate setting the alias "Champion" to the best-performing model and getting the latest model version to be deployed.

```
from mlflow import MlflowClient
import mlflow

mlflow.set_registry_uri('databricks-uc')
client = MlflowClient()
model_name = f'{catalog}.{database_name}.cvops_model_mlaction'

client.set_registered_model_alias(model_name, "Champion", 1)

model_version_uri = f"models:/{model_name}@Champion"
# Or another option: model_version_uri = f"models:/{model_name}/1"
loaded_model_uc = mlflow.pyfunc.load_model(model_version_uri) # runiid / model_registry_name+version
# champion_version = client.get_model_version_by_alias("prod.ml_team.iris_model", "Champion")
latest_model = client.get_model_version_by_alias(model_name, "Champion")
```

Figure 7.35 – Loading our champion model